

Asaan Rishta Developer Handover Requirements

Repository-based current-state specification

Prepared for Flutter developers, backend owners, QA, and product owners

Source snapshot reviewed 30 September 2026

This document maps the implemented client and supporting Firebase code. It is a handover baseline, not a claim that every flow works in production. A statement marked **Verified in code** identifies an implementation path. Backend outcomes, live payment settlement, and platform behavior need separate confirmation. Open questions are kept apart from implementation facts.

1. Executive overview and evidence rules

Asaan Rishta is a Flutter matrimonial application with public browsing, ordinary user and matrimonial account paths, profile discovery, vendors, chat, connects, payments, and account tools. The client uses GetX for routes and controllers, HTTP repositories for a separate backend, and Firebase for authentication support, chat, messaging, storage, notifications, and functions. Local state includes secure storage, SharedPreferences, and Hive message cache.

Sources: `lib/main.dart:45-140`; `lib/app/core/routes/app_pages.dart`; `lib/app/core/services/dependency_injection.dart`; `pubspec.yaml`.

Evidence labels. A = Verified current client behavior in named code; B = documented intention with no independent implementation confirmation; C = proposed behavior or unresolved question. All module descriptions below are A unless explicitly marked B or C. A cited call establishes that the client attempts an operation, not that the remote system accepts it.

2. Repository and architecture

Inventory. The repository contains the Flutter application under *lib*; assets; Android and iOS projects; Dart unit, widget, and integration tests; JavaScript Firebase callable functions under *functions*; Fastlane files; environment examples; and several issue-specific Markdown notes. Routes and bindings live under *lib/app/core*. UI is under *lib/app/views*, controllers under *lib/app/viewmodels* and some view folders, domain use cases and repositories under *lib/app/domain*, and chat storage under *lib/app/data* and core services. Sources: `lib/app/core/routes/app_pages.dart`; `lib/app/core/services/dependency_injection.dart`; `functions/index.js`; `test/`; `integration_test/`.

Startup. Main loads `.env`, initializes secure storage and Hive, initializes Firebase and App Check, registers repositories/use cases, installs a Firebase background message handler, initializes deep links and screen security, and chooses onboarding or splash from stored onboarding state. Initialization errors for environment, secure storage, and Hive are logged but startup continues. Splash waits for AuthService and routes signed-in users to bottom navigation or partner preferences, with role 3 bypassing preferences; guests go to account type. Sources: `lib/main.dart:45-140`; `lib/app/views/splash/splash_controller.dart:24-112`.

Technology. Dart SDK constraint ^3.12.1; GetX, dartz, HTTP and Dio, Firebase Core/Auth/Firestore/Storage/Messaging/App Check/Functions/Remote Config, SharedPreferences, flutter_secure_storage, Hive, in_app_purchase, WebView, media and notification plugins. Package declarations establish dependencies, not proof every integration is active. Source: `pubspec.yaml:53-174`.

Data ownership. The backend API supplies registration, profile/catalogue data, favorites, connects, transactions, vendors, and contact operations. Firestore stores chat users, conversations, presence, preferences flags, and notification data. Hive caches messages; secure storage and SharedPreferences hold account/session values. The client combines data from these systems; no shared cross-system transaction is visible. Sources:

`lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart`;
`lib/app/domain/repository/system_config_repo/system_config_repo_impl.dart`;
`lib/app/core/services/firebase_service/firebase_service.dart`; `lib/app/core/services/hive_message_service.dart`;
`lib/app/core/services/secure_storage_service.dart`.

3. Roles, access and navigation

State	Visible bottom tabs	Evidence and limits
Guest	Home, Vendors, Filter, Account	Guest pages are explicitly selected when not logged in. Individual screen actions may prompt login. lib/app/viewmodels/bottom_nav_viewmodel.dart:86-116.
Signed-in ordinary user	Home, Vendors, Chats, Filter, Profile	Profile and chat tabs are in the ordinary page list. Same source.
Role ID 3 matrimonial account	Home, Vendors, Chats, Filter, Dashboard	Dashboard replaces Profile; login and splash bypass partner preference. lib/app/viewmodels/bottom_nav_viewmodel.dart:94-116; lib/app/viewmodels/login_viewmodel.dart:162-263; lib/app/views/splash/splash_controller.dart:75-94.

Navigation map. First install → onboarding → account type → ordinary signup or matrimonial signup, login, or guest tabs. Login → token → donor claims → role validation → dashboard/bottom tabs or partner preferences. Bottom tabs lead to Home → user details → chat; Vendors → category → list → details; Chats → chat list → conversation; Filter → results → user details; Profile or Dashboard → account, purchases, histories, managed profiles, and support routes. Named routes also include password recovery, privacy/terms WebViews, and a dynamic chat route. The account-deactivated route is declared but its page registration is commented out. Sources: lib/app/core/routes/app_routes.dart:1-58; lib/app/core/routes/app_pages.dart:36-315; lib/app/views/profile/profile_view.dart:328-505.

4. Module requirements and acceptance criteria

M01 Onboarding, account choice and guest browsing

Entry and actions. Onboarding is selected at first run; account choice offers Rishta User, Matrimonial, login, guest, user guide, and contact. Selected account type is saved before login. Guest goes to reduced bottom tabs. **Inputs/rules.** Account type selection and stored onboarding flag govern route choice. **Failure.** Startup may proceed after initialization catches; user feedback depends on screen. Sources: lib/main.dart:128-140; lib/app/viewmodels/account_type_viewmodel.dart:11-96; lib/app/views/account_type/account_type_view.dart:16-130.

Acceptance check: Fresh install shows onboarding; returning guest reaches four tabs; selecting each type opens its corresponding form; login remembers the selected type for role validation.

M02 Ordinary user registration

Entry and actions. Signup form proceeds through basic information and address/preferences; the controller loads castes, degrees, occupations, countries, states and cities, handles photo selection, validates fields, and submits a backend registration. A dashboard-created profile path and approval message exist. **Inputs.** Name, email, phone, password, gender, date of birth, photo, education/occupation/caste, address and preference selections are represented across the form/controller. Exact backend approval criteria are unverified. **Failure.** Validation and repository errors surface in controller UI. Sources: lib/app/views/signup/signup_view.dart:22-360; basic_info_view.dart:12-140; address_preferences_view.dart:14-110; lib/app/viewmodels/signup_viewmodel.dart:31-1194; lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart:96-149.

Acceptance check: Invalid required values block progress; dependent location options load in order; completed form sends one registration request; failure remains actionable; dashboard-created users follow the distinct return path.

M03 Matrimonial registration and managed profiles

Entry and actions. Matrimonial signup uses VendorRegistrationViewModel and the vendor registration API. Role 3 dashboard displays account information and routes to add a user or view managed profiles. Managed profiles can be inspected, their images updated, and a delete action is presented. **Rules.** The role 3 dashboard replaces the ordinary Profile tab. Server authority for managed-profile deletion needs backend confirmation. Sources:

lib/app/views/signup/matrimonial_signup_view.dart:22-607; lib/app/viewmodels/vendor_registration_viewmodel.dart:18-503;
lib/app/views/dashboard/dashboard_view.dart:485-511; lib/app/viewmodels/matrimonial_profiles_viewmodel.dart:34-465;
lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart:985-1088.

Acceptance check: Role 3 sees Dashboard; managed profile list loads for the admin ID; add, image update and delete paths display API failures clearly.

M04 Login, session and logout

Flow. Login posts username/password/grant_type to token endpoint, stores token locally, fetches donor claims with bearer authorization, checks selected account type against role ID, saves session and creates/updates a chat user. AuthService checks stored session at startup, handles Firebase authentication, FCM token and logout cleanup.

Validation. Email required and syntactically checked; password minimum length four in the login controller. **Failure.** Token and claims errors are shown separately; the repository sets a SharedPreferences login flag after token success before claims succeed. Sources: lib/app/viewmodels/login_viewmodel.dart:52-285;

lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart:60-189;
lib/app/viewmodels/auth_service.dart:81-273.

Acceptance check: Invalid credentials do not navigate; role mismatch is blocked; successful role-appropriate session reaches the correct route; logout clears visible user state. Confirm partial-token failure behavior with product/backend owner.

M05 Password recovery and password change

Flow. Forgot password loads a phone number, requests Firebase phone verification, accepts or pastes an OTP, and after verification calls backend reset_password with the email and new password. Signed-in change password uses the backend update password operation. **Failure.** OTP and reset failures are handled in controller paths; account-to-phone mapping must be confirmed server-side. Sources: lib/app/views/forgot_password/forgot_password_controller.dart:100-489;

lib/app/views/profile/change_password/change_password_controller.dart:28-79;
lib/app/core/services/network_services/end_points.dart:52-60.

Acceptance check: OTP request/verification and resend states are visible; invalid OTP cannot advance; password reset and signed-in change display success/failure consistently.

M06 Home, featured profiles and profile details

Flow. Home fetches paged ordinary or featured profiles, filters current user and opposite-gender profiles in controller logic, opens user details, and can add favorites. User details fetches profile information, checks connects and chat relationship, and offers media previews and communication actions. Guest action gates are present in controller logic.

Failure. Loading/error UI is controller-dependent; backend profile visibility and gender policy need confirmation.

Sources: lib/app/viewmodels/home_viewmodel.dart:37-236; lib/app/views/home/home_view.dart:43-170;
lib/app/viewmodels/user_details_viewmodel.dart:23-190;
lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart:588-646.

Acceptance check: Pagination does not duplicate profiles; card opens correct ID; guest actions follow login rules; failed detail load is visible without stale profile data.

M07 Filter and profile search

Flow. Filter offers featured toggle, age, gender, marital status, religion, caste, education/occupation and location selectors; controller sends backend filter calls and shows matching cards. Location values use country→state→city lists. **Failure.** Controller clears or updates results on error; exact server matching semantics are unverified. Sources: lib/app/views/filter/filter_view.dart:348-592; lib/app/viewmodels/filter_viewmodel.dart:27-425; lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart:648-724.

Acceptance check: Selected filters produce expected request fields; clearing filters restores results; pagination and featured mode do not mix incompatible result sets.

M08 Own profile, editing, photo and blur

Flow. Profile menu opens details, edit, partner preferences, favorites, connects purchase/history, transactions, password, support and guides. ProfileController loads API profile data, uploads image data, synchronizes chat image, and exposes an optimistic blur setting update through the backend. Role 3 has a separate vendor edit route. **Failure.** Blur update rolls back on error; image upload and Firestore sync can fail separately. Sources: lib/app/views/profile/profile_view.dart:328-528; lib/app/viewmodels/profile_viewmodel.dart:533-850; lib/app/views/profile/edit_profile/edit_profile_controller.dart; vendor_edit_profile_controller.dart.

Acceptance check: Saved edits reload from API; photo appears in profile and chat after sync; blur toggle reverts after failed API response; role-appropriate edit form opens.

M09 Partner preferences

Flow. Preference screen loads lookup lists and existing preferences, offers multi-select and location fields, sends a backend update, and participates in Firestore is_preference_updated routing on login/splash. **Rule.** Role 3 bypasses this screen. Sources: lib/app/views/profile/partner_preference/partner_preference_view.dart:13-672; partner_preference_controller.dart:15-454; lib/app/views/splash/splash_controller.dart:75-94.

Acceptance check: Saved values reload; dependent choices remain valid; ordinary user reaches app after completion; role 3 bypasses preferences.

M10 Favorites

Flow. Home and profile details use add-to-favorites API; Favorites screen loads a backend favorites list and opens profiles. Exact toggle/remove semantics must be checked in backend responses. Sources: lib/app/views/home/home_view.dart:169-170; lib/app/views/profile/favorites/favorites_controller.dart; lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart:897-954; lib/app/core/services/network_services/end_points.dart:128-133.

Acceptance check: Add operation updates visible state and list after reload; failures do not falsely show a saved favorite.

M11 Chat list, conversations and cache

Flow. Signed-in chat tab lists Firestore chat users and conversation previews; conversation view uses ChatViewModel, ChatRepository and FirebaseService for real-time messages, status, typing, reads, deletion, blocking and media. HiveMessageService stores local messages. The active chat implementation is in chatting_view.dart; chatting_controller.dart and chatting_view_viewmodel.dart contain commented legacy code. Message ID/filter utilities and recent tests exist. **Data.** Firestore chat collections are configured through EnvConfig; Firebase Storage handles images. **Failure.** Cached and remote messages, async switches and deletes require device validation. Sources: lib/app/views/chat/chat_user_listing_view.dart; chatting_view.dart; lib/app/viewmodels/chat_viewmodel.dart:701-743; chat_list_viewmodel.dart:408-996; lib/app/data/repositories/chat_repository.dart:121-462,901-1006; lib/app/core/services/hive_message_service.dart; firebase_service/firebase_service.dart:684-895.

Acceptance check: Switching chats never shows another conversation; rapid sends retain unique messages; restart reloads only the correct account/chat cache; deleted-chat reopen shows the first new message; block/read/delivery states update safely.

M12 Connects deduction and balance

Flow. Backend API exposes current connect count, purchase credit update and deduction by recipient. First-message chat flow also invokes FirebaseService deduction logic; user details checks existing connection. **Limit.** Code shows client-side checks and calls, but exactly-once deduction across concurrent sends, retries and two data stores is not established by the repository. No server transaction contract is available. Sources:

lib/app/domain/repository/system_config_repo/system_config_repo_impl.dart:223-334;

lib/app/core/services/network_services/end_points.dart:136-145; lib/app/core/services/firebase_service/firebase_service.dart:777-895;

lib/app/viewmodels/user_details_viewmodel.dart:23-115.

Acceptance check proposed for owner confirmation: One charge per new recipient even on retry/restart, and balance/ledger agree with the delivered first message. This is C, not a verified current guarantee.

M13 Connects purchase and payment methods

Flow. Buy Connects offers packages, Google Play in-app consumables, PayFast WebView path and manual payment information with WhatsApp receipt contact. Purchase listener handles purchased, pending, restored, error and canceled statuses; it calls backend credit/transaction operations after its verification path. PayFast return refreshes connects. **Limit.** In-memory processed-purchase sets are not proof of durable server-side idempotency. Payment settlement, receipt verification and store policy need payment owner confirmation. Sources:

lib/app/views/profile/buy_connects/buy_connects_view.dart:14-423; buy_connects_controller.dart:22-540,669-901;

lib/app/fast_pay/views/webview_screen.dart:41-256; lib/app/domain/repository/system_config_repo/system_config_repo_impl.dart:274-413.

Acceptance check: Pending/error/cancel show distinct states; backend credits match a verified purchase once; app restart/replay behavior must be tested with payment sandbox and backend ledger.

M14 Transaction and connects histories

Flow. Profile opens backend transaction and connection-history lists; controllers generate downloadable PDF reports.

Failure. Empty/error lists and file-open behavior depend on backend/platform. Sources:

lib/app/views/profile/transaction_history/transaction_history_controller.dart:22-158;

connects_history/connects_history_controller.dart:23-162;

lib/app/domain/repository/system_config_repo/system_config_repo_impl.dart:414-480.

Acceptance check: Each list matches backend records and generated PDF shows the same entries and totals; empty state and export errors are visible.

M15 Vendors, listings and details

Flow. Vendor tab displays local category cards, category opens paged/city-filtered backend list, and details fetch services, questions, albums, videos and packages. Video player is a separate screen. Matrimonial registration/profile edit use related but distinct APIs. **Failure.** Independent detail requests may partly fail; list controller tracks loading and pagination. Sources: lib/app/views/vendor/vendor_view.dart:13-85; vendor_controller.dart:17-29;

listing/vendor_listing_controller.dart:39-90; details/vendor_detail_controller.dart:106-315;

lib/app/domain/repository/user_management_repo/user_management_repo_impl.dart:726-896.

Acceptance check: Category and city selections reach backend, list opens correct vendor, each detail section handles empty/error data, media playback can be exited safely.

M16 Notifications, links and presence

Flow. FCM background, foreground and tap listeners are registered; notification service stores/removes token and navigates to chat. Deep links are initialized at startup and pending links are processed after splash. Firebase chat/presence status is updated by AuthService and bottom navigation/lifecycle logic. Callable functions confirm delivery and mark pending messages. **Limit.** Delivery function catches some errors and returns success to suppress retries; this is a code behavior, not confirmed delivery. Sources: lib/main.dart:123-125,145-175;

lib/app/core/services/firebase_service/notification_service.dart:291-352,542-579,1025-1050;

lib/app/core/services/deep_link_handler.dart; lib/app/viewmodels/bottom_nav_viewmodel.dart:136-325; functions/index.js:6-277.

Acceptance check: Foreground/background/tapped notifications route to intended recipient; missing user data falls back safely; read/delivery indicators reflect actual Firestore state; lifecycle presence recovers on resume.

M17 Support, informational and account lifecycle

Flow. The active Contact Us controller opens external email and WhatsApp intents; a separate backend contact method exists in the repository, but this screen does not call it. About Us, user guide/FAQs, and privacy/terms WebViews are routed. Profile menu offers logout, deactivate and delete, with Firebase cleanup paths. **Limit.** Backend retention/deletion effects and external website contents are outside this repository. Sources:

lib/app/views/profile/contact_us/contact_us_controller.dart:34-157;
lib/app/domain/repository/system_config_repo/system_config_repo_impl.dart:482-515;
lib/app/views/profile/about_us/about_us_view.dart; user_guide/user_guide.dart; lib/app/views/profile/profile_view.dart:540-802;
lib/app/viewmodels/profile_viewmodel.dart:372-502,752-782; lib/app/core/routes/app_pages.dart:185-211.

Acceptance check: Contact failures are explicit; external intents and WebViews open correctly; logout clears account views; deactivation/deletion confirmation and backend effects are verified separately.

M18 Security, configuration and platform support

Flow. .env supplies API/Firebase/PayFast settings; Firebase options are generated; app enables screen protection and App Check (Android debug or Play Integrity). Android declares internet, notifications, boot/vibration/wake, ad ID and overlay permissions; iOS declares camera/microphone/photo usage and background modes. **Limit.** Runtime permission grants and production signing are not verifiable from code alone. An asset-loading service-account path is present and must be reviewed before any production release. Sources: lib/app/core/services/env_config_service.dart:17-147; lib/main.dart:45-123; pubspec.yaml:187-204; android/app/src/main/AndroidManifest.xml:2-9; ios/Runner/Info.plist:43-59; lib/app/core/utis/screen_security.dart.

Acceptance check: Missing config yields a controlled startup state; platform permission prompts match features; no server private key is packaged in client builds; release builds use approved signing and Firebase configuration.

5. End-to-end journeys and data flow

- Ordinary signup:** Account type → signup/basic/address → lookup API → registration API → approval/return state → login → claims → preference completion → bottom tabs. Source: signup_viewmodel.dart:933-1090; login_viewmodel.dart:52-263.
- Matrimonial account:** Account type → matrimonial form → vendor registration API → login role 3 → dashboard → managed profiles → add/update/remove. Source: vendor_registration_viewmodel.dart:419-503; bottom_nav_viewmodel.dart:86-116; matrimonial_profiles_viewmodel.dart:63-465.
- Discover/contact:** Home or Filter → backend profile list → detail API → connection check/balance → Firestore chat user/conversation → message Firestore/Storage and Hive cache → delivery function/FCM. Connect debit also calls backend; atomicity across systems is unverified. Source: home_viewmodel.dart:93-203; user_details_viewmodel.dart; chat_repository.dart; firebase_service.dart.
- Buy connects:** Profile → Buy Connects → package → Google Play / PayFast / manual instructions → backend credit and transaction calls where implemented → balance/history refresh. External settlement must be confirmed. Source: buy_connects_controller.dart:190-540,669-901; system_config_repo_impl.dart:274-480.
- Return or logout:** Startup secure storage/session → splash role/preference decision → FCM/presence; logout cleans session and chat state. Source: main.dart:45-140; splash_controller.dart:24-115; auth_service.dart:195-273.

6. Integrations, configuration and build

Integration	Observed contract	Confirmation needed
Backend HTTP API	BASE_URL and AUTH_BASE_URL; token, claims, profile, reference lists, vendors, connects and transaction endpoints. lib/app/core/services/network_services/end_points.dart:1-213.	OpenAPI/schema, authorization rules, error semantics, availability, idempotency and test environment.
Firebase	Core, Auth, Firestore, Storage, Messaging, App Check, Remote Config and callable Functions referenced in client/dependencies. lib/main.dart;	Rules/indexes, project access, iOS/Android notification setup

	functions/index.js.	and deployment state.
Payments	Google Play IAP, PayFast WebView/config, manual receipt contact. buy_connects_controller.dart.	Store products, server receipt verification, ledger idempotency and reconciliation.
Local data	Secure storage, SharedPreferences, Hive message cache. main.dart:57-88; secure_storage_service.dart; hive_message_service.dart.	Retention, migration, encryption and account-switch policy.

Environment variable names observed: BASE_URL, AUTH_BASE_URL; FIREBASE_API_KEY, FIREBASE_IOS_API_KEY, FIREBASE_APP_ID, FIREBASE_IOS_APP_ID, FIREBASE_MESSAGING_SENDER_ID, FIREBASE_PROJECT_ID, FIREBASE_IOS_BUNDLE_ID, FIREBASE_STORAGE_BUCKET, FIREBASE_USERS_COLLECTION, FIREBASE_CHATS_COLLECTION; PAYFAST_CHECKOUT_URL, PAYFAST_TRANSACTION_URL, PAYFAST_SUCCESS_URL, PAYFAST_FAILURE_URL, PAYFAST_MERCHANT_ID; DEFAULT_PROFILE_IMAGE; FIREBASE_SERVICE_ACCOUNT_* fields. No values or secrets are reproduced here. .env.example does not list every getter in EnvConfig. Sources: lib/app/core/services/env_config_service.dart:38-97; .env.example.

Build prerequisites visible: Flutter/Dart matching pubspec constraint, Android and iOS native toolchains, Firebase platform configuration, .env asset, Android signing and iOS provisioning, Firebase functions Node dependencies. Android target SDK is 36; min SDK uses Flutter default. iOS permission strings and background modes are in Info.plist. Fastlane files exist for Android. Exact build success was not verified because Flutter/Dart executables are unavailable in this workspace session. Sources: pubspec.yaml:53-204; android/app/build.gradle.kts:30-40; ios/Runner/Info.plist:43-59; android/fastlane/Fastfile; functions/package.json.

7. Existing tests and verification limits

The repository has unit tests for account type, navigation, login, signup, home, filter, profile, user details and chat; chat utility/cache tests; widget tests for major screens; and two integration test files. Presence of a test is not evidence it passes. Flutter/Dart are not on PATH here, so no Dart test or build execution was performed for this handover. There is no visible dedicated automated coverage for live backend claims failure, purchase recovery across restart, cross-store connects deduction, Firebase rules, push delivery on physical devices, or complete account deletion. Sources: test/unit/; test/widget/; integration_test/app_test.dart; integration_test/complete_user_flows_test.dart.

8. Open questions and gaps

- Backend claims after token:** The client calls GetDonorClaims after a successful token and treats a 500 as error. Backend logs/schema are required to explain failures; do not define this as expected behavior. login_viewmodel.dart:111-155; user_management_repo_impl.dart:152-189.
- Connects atomicity:** Confirm exact debit trigger, refund/retry behavior, recipient key and server transaction/idempotency contract across backend and Firestore. firebase_service.dart:777-895; system_config_repo_impl.dart:307-334.
- Purchase fulfillment:** In-memory duplicate guards do not survive process death; confirm server receipt validation, unique transaction IDs and restore handling. buy_connects_controller.dart:228-456.
- Approval and roles:** Define who approves ordinary and matrimonial accounts, status lifecycle, and backend authorization for role 3 management. Client labels alone do not establish server permissions. signup_viewmodel.dart:1014-1090; login_viewmodel.dart:162-193; matrimonial_profiles_viewmodel.dart:300-323.
- Client secrets:** EnvConfig reads service-account fields and an asset JSON path. Confirm no service-account private key is shipped to the app and rotate any exposed credential. env_config_service.dart:26-27,73-121; pubspec.yaml:187-204.
- Account deletion:** Confirm backend data retention, chat partner visibility and consistency of Firestore cleanup. profile_viewmodel.dart:372-502; profile_view.dart:639-739.
- Delivery semantics:** Callable function may return success after an internal error; define whether that is acceptable and how undelivered messages are repaired. functions/index.js:6-215.
- Platform/build:** Confirm iOS notification entitlements/provisioning, Android overlay permission need, release signing, and actual store product configuration. ios/Runner/Runner.entitlements;

android/app/src/main/AndroidManifest.xml; buy_connects_controller.dart.

- 9. Documentation-only assertions:** Existing Markdown fix/test notes are B until checked against compiled behavior. The account-deactivated route has no active GetPage registration. lib/app/core/routes/app_pages.dart:215-223; root Markdown notes.

9. Traceability appendix

Module	Primary source	Relevant visible tests
M01	main.dart; account_type_viewmodel.dart; splash_controller.dart	account_type_viewmodel_test.dart; bottom_nav_view_test.dart
M02	signup_viewmodel.dart; views/signup/*; user_management_repo_impl.dart	signup_viewmodel_test.dart
M03	vendor_registration_viewmodel.dart; dashboard_viewmodel.dart; matrimonial_profiles_viewmodel.dart	No dedicated managed-profile test located
M04	login_viewmodel.dart; auth_service.dart; user_management_repo_impl.dart	login_viewmodel_test.dart; login_view_test.dart
M05	forgot_password_controller.dart; change_password_controller.dart	No dedicated recovery test located
M06	home_viewmodel.dart; user_details_viewmodel.dart	home_viewmodel_test.dart; user_details_viewmodel_test.dart; home_view_test.dart
M07	filter_viewmodel.dart; filter_view.dart	filter_viewmodel_test.dart; filter_view_test.dart
M08-M10	profile_viewmodel.dart; partner_preference_controller.dart; favorites_controller.dart	profile_viewmodel_test.dart; profile_view_test.dart
M11-M12	chat_viewmodel.dart; chat_list_viewmodel.dart; chat_repository.dart; firebase_service.dart; hive_message_service.dart	chat_viewmodel_test.dart; chat_session_reset_test.dart; hive_message_service_test.dart; chat_message_id_test.dart; chat_message_filter_test.dart; chat_view_test.dart
M13-M14	buy_connects_controller.dart; webview_screen.dart; history controllers	No dedicated purchase/history test located
M15	views/vendor/*; user_management_repo_impl.dart	vendor_details_view_test.dart
M16-M18	notification_service.dart; deep_link_handler.dart; functions/index.js; env_config_service.dart; platform manifests	Integration files exist; no targeted server/platform test located

Source references use repository-relative paths and line numbers as observed on 30 September 2026. Files marked with * in the appendix are directory groups. This specification does not copy .env values, API tokens, credentials, personal data, or service-account material.